

LOGSIGHT: ADAPTIVE MULTI-VIEW LOG ANOMALY DETECTION AND ROOT- CAUSE LOCALIZATION

PROJECT REPORT

Submitted by

JEGADEESWARAN D

2116231801069

KARTHICK S

2116231801079

KOUSHAL V

2116231801084

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND
DATA SCIENCE**



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

**(An Autonomous Institution, Affiliated to Anna
University, Chennai)**

SEPTEMBER 2026



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this mini-project report “**Logsight: Adaptive Multi-View Log Anomaly Detection And Root-Cause Localization**” is the bonafide work of “**Jegadeeswaran D (231801069), Karthick S (231801079), Koushal V (231801084)**”, who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. J.M. Gnanasekar M.E., Ph.D.,
Professor and Head,

Department of Artificial Intelligence and
Data Science

Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Ms. Jayasri Archana Devi M.E.,
Assistant Professor (SG),

Department of Artificial Intelligence and
Data Science

Rajalakshmi Engineering College
Thandalam – 602 105

Submitted to mini-project viva voce examination held on:

Internal Examiner

External Examiner

DEPARTMENT VISION

To become a global leader in Artificial Intelligence and Data Science by achieving through excellence in teaching, training, and research, to serve the society.

DEPARTMENT MISSION

- To develop students' skills in innovation, problem-solving, and professionalism through the guidance of well-trained faculty.
- To encourage research activities among students and faculty members to address the evolving challenges of industry and society.
- To impart qualities such as moral and ethical values, along with a commitment to lifelong learning

PROGRAMME EDUCATIONAL OBJECTIVES (PEO's)

PEO 1: Build a successful professional career across industry, government, and academia by leveraging technology to develop innovative solutions for real-world problems.

PEO 2: Maintain a learning mindset to continuously enhance knowledge through experience, formal education, and informal learning opportunities.

PEO 3: Demonstrate an ethical attitude while excelling in communication, management, teamwork, and leadership skills

PEO 4: Utilize engineering, problem-solving, and critical thinking skills to drive social, economic, and sustainable impact.

PROGRAMME OUTCOME (PO's)

PO1: Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design / Development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and team work: Function effectively as an individual and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change

PROGRAM SPECIFIC OUTCOMES (PSOs)

A graduate of the Artificial Intelligence and Data Science Learning Program will demonstrate

PSO 1: Foundation Skills: Apply the principles of artificial intelligence and data science by leveraging problem-solving skills, inference, perception, knowledge representation, and learning techniques

PSO 2: Problem-Solving Skills: Apply engineering principles and AI models to solve real-world problems across domains, delivering cutting-edge solutions through innovative ideas and methodologies

PSO 3: Successful Progression: Utilize interdisciplinary knowledge to identify problems and develop solutions, a passion for advanced studies, innovative career pathways to evolve as an ethically responsible artificial intelligence and data science professional, with a commitment to society.

COURSE OBJECTIVE

- To identify and formulate real-world problems that can be solved using Artificial Intelligence and Data Science techniques.
- To apply theoretical and practical knowledge of AI & DS for designing innovative, data-driven solutions.
- To integrate various tools, frameworks, and algorithms to develop, test, and validate AI & DS models.
- To demonstrate effective teamwork, project management, and communication skills through collaborative project execution.
- To instill awareness of ethical, societal, and environmental considerations in the design and deployment of intelligent systems.

COURSE OUTCOME

CO 1: Analyze and define a real-world problem by identifying key challenges, project requirements and constraints.

CO 2: Conduct a thorough literature review to evaluate existing solutions, identify research gaps and formulate research questions.

CO 3: Develop a detailed project plan by defining objectives, setting timelines, and identifying key deliverables to guide the implementation process.

CO 4: Design and implement a prototype or initial model based on the proposed solution framework using appropriate AI tools and technologies.

CO 5: Demonstrate teamwork, communication, and project management skills by preparing and presenting a well-structured project proposal and initial implementation results.

CO-PO-PSO Mapping

C O	P O1	P O2	P O3	P O4	P O5	P O6	P O7	P O8	P O9	PO 10	PO 11	PO 12	PS O1	PS O2	PS O3
C O1	3	3	2	2	1	2	1	1	1	2	1	2	3	2	2
C O2	2	3	2	3	2	1	1	1	2	2	1	3	2	2	2
C O3	2	2	3	2	2	1	2	2	3	2	3	2	2	3	3
C O4	3	3	3	3	3	2	2	2	2	3	2	2	3	3	3
C O5	2	2	2	1	2	2	2	3	3	3	3	2	2	2	3

Note: Correlation levels 1, 2 or 3 are as defined below:

1: Slight (Low)

2: Moderate (Medium)

3: Substantial (High)

No correlation: “-”

ABSTRACT

Large computing systems produce millions of log lines, and failures hide inside them as unusual messages, rare event types, or abnormal bursts and orderings of events. Labelled failure data is rarely available, and rule-based monitoring catches only the problems someone anticipated. This project presents LogSight, an unsupervised multi-view system that detects anomalies in system logs, explains each detection with evidence, groups detections into incidents, and ranks the components most likely to be their origin. It is built and evaluated on the BlueGene/L (BGL) supercomputer log. LogSight examines each log line through three views: a semantic view, in which Sentence-BERT embeddings are compared against KMeans prototypes of normal messages; a structural view, in which template, level, component, parameters, and template rarity are scored by Isolation Forest and Local Outlier Factor; and a temporal view, in which a Hidden Markov Model scores event order and a rolling z-score scores event frequency. The three scores are combined by reliability-weighted fusion, where each view's weight is set per line from its reliability and scaled by how informative the view is on the unlabelled learning window.

A rolling median + MAD threshold flags anomalies, and each flag receives a severity level. Flags are grouped into incidents, candidate root-cause components are ranked, each flag receives an evidence package, and a Kolmogorov-Smirnov test with a shuffle control monitors drift. On the full BGL log (4,713,483 cleaned lines, 7.39% labelled anomalies, 1,819 templates), the research pipeline achieves an AUC-ROC of 0.7237 and an AUC-PR of 0.1900 on a chronological test split of 942,697 lines. The AUC-ROC rises from 0.456 once a data-leakage bug and a weak view are diagnosed and corrected. In the ablation study, equal-weight fusion (0.470) performs worse than the best single view (0.830), and weighted fusion recovers most of the gap (0.7237) but does not exceed the structural view alone.

On test lines whose templates never appear in training, the AUC-ROC is 0.658. The pipeline is delivered as a full-stack application with a FastAPI backend, SQLite storage, 22 REST endpoints, and a React dashboard that shows live run progress, timelines, incident cards, a root-cause cluster map, and per-line attribution. In the application, an improved semantic view raises its share of the fusion weight from 11.8% to 38.2% on a 50,000-line sample, without using any labels.

Keywords: system logs, log anomaly detection, unsupervised learning, multi-view learning, Sentence-BERT, Isolation Forest, Local Outlier Factor, Hidden Markov Model, score fusion, root-cause localization, explainable AI, BGL

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J.M. GNANASEKAR M.E., Ph.D.**, Professor and Head of the Department of Artificial Intelligence and Data Science for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Ms. JAYASRI ARCHANA DEVI M.E.**, Assistant Professor (SG), Department of Artificial Intelligence and Data Science. Rajalakshmi Engineering College for her valuable guidance throughout the course of the project. We are very glad to thank our Project Coordinator, **Ms. JAYASRI ARCHANA DEVI M.E.**, Department of Artificial Intelligence and Data Science for his useful tips during our review to build our project.

Jegadeeswaran D
Karthick S
Koushal V

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT	7
	ACKNOWLEDGEMENT	8
	LIST OF FIGURES	10
	LIST OF TABLES	11
	LIST OF ABBREVIATIONS	11
1	INTRODUCTION	12
	1.1 General	12
	1.1.1 System Logs in Large-Scale Computing	12
	1.1.2 Log Anomaly Detection	12
	1.1.3 Unsupervised Learning for Logs	12
	1.1.4 Multi-View Representation of Log Events	13
	1.1.5 Explainability and Root-Cause Localization	13
	1.2 Objectives	13
	1.3 Existing System	13
	1.4 Proposed System	14
2	LITERATURE SURVEY	15
	2.1 Overview	15
	2.2 Literature Survey	15
	2.2.1 Log Parsing and Datasets	15
	2.2.2 Statistical and Sequence-Based Detection	15
	2.2.3 Semantic and Transformer-Based Methods	16
	2.2.4 Outlier Detection and Sequence Likelihood	16
	2.2.5 Robust Thresholding and Drift	17
	2.2.6 Multi-System Detection	17
	2.3 Base Paper	17
	2.4 Research Gap	18
3	SYSTEM DESIGN	19
	3.1 Dataset Loading	19
	3.2 Development Environment	19
	3.3 Architecture	20
	3.4 Multi-View Detection Model	20
	3.5 Fusion, Threshold and Severity	21
	3.6 Explainability, Drift and Root-Cause Module	21
	3.7 Backend and Database Module	22
	3.8 Visualization Module	22

4	METHODOLOGY	24
	4.1 Data Collection and Preprocessing	24
	4.2 Log Parsing and Exploratory Analysis	25
	4.3 Multi-View Feature Engineering	26
	4.4 Anomaly Scoring and Reliability-Weighted Fusion	27
	4.5 Adaptive Thresholding, Severity and Drift	28
	4.6 Evidence Generation and Root-Cause Localization	28
	4.7 Application Integration	29
	4.8 Evaluation Protocol	29
5	RESULTS AND DISCUSSION	31
	5.1 Overview	31
	5.2 Functional Verification	31
	5.3 Dataset and Parsing Results	31
	5.4 Detection Performance Analysis	32
	5.5 Ablation and Unseen-Template Results	32
	5.6 Threshold and Drift Results	34
	5.7 Application Results	34
	5.8 Dashboard Results	35
	5.9 Comparative Discussion	37
	5.10 Discussion and Insights	38
6	CONCLUSION AND FUTURE ENHANCEMENTS	40
	6.1 Conclusion	40
	6.2 Future Enhancements	40
	REFERENCES	42

LIST OF FIGURES

Figure No.	Title
3.1	System architecture of LogSight
4.1	Hourly log volume of BGL
4.2	Top 15 components by log volume
5.1	Overview dashboard
5.2	Incidents page: 2D root-cause cluster map
5.3	Flagged line detail with Shapley attribution

LIST OF TABLES

Table No.	Title
4.1	Research pipeline (v1) versus application (v2)
5.1	Dataset, parsing and split statistics
5.2	AUC-ROC progression across corrections
5.3	Final research test metrics (v1, $\lambda = 1.5$)
5.4	Ablation results
5.5	Unseen vs seen template results (test split)
5.6	Threshold comparison on the test split
5.7	Semantic view v1 vs v2 (BGL sample 1)
5.8	Feature comparison with existing approaches

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
AUC-PR	Area Under the Precision-Recall Curve
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
BERT	Bidirectional Encoder Representations from Transformers
BGL	BlueGene/L
HMM	Hidden Markov Model
IF	Isolation Forest
JSON	JavaScript Object Notation
KS	Kolmogorov-Smirnov
LLM	Large Language Model
LOF	Local Outlier Factor
LSTM	Long Short-Term Memory
MAD	Median Absolute Deviation
ML	Machine Learning
PCA	Principal Component Analysis
RCA	Root-Cause Analysis
REST	Representational State Transfer
SBERT	Sentence-BERT
SQL	Structured Query Language
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 General

Computing systems record their activity in logs. Each line holds a timestamp, the component that wrote it, a severity level and a free-text message. When something goes wrong, logs are usually the first and often the only evidence. The BGL log used here has 4,747,963 raw lines collected over about seven months, which is far too much to read by hand. Deciding which lines matter, why, and where a problem began requires automation.

1.1.1 System Logs in Large-Scale Computing

Supercomputers, distributed storage and cloud services consist of many nodes that log independently. Messages are semi-structured: a fixed part names the event type, and a variable part carries addresses, counters and identifiers. Logs are also uneven in time. In BGL, half of all consecutive lines are less than 26 ms apart, so failures often appear as dense bursts rather than isolated lines.

A raw BGL line has ten fields: label (- for normal, an alert category otherwise, used for evaluation only), Unix timestamp, date, node, full microsecond time (the true ordering key), repeated node, type, component, level and content.

1.1.2 Log Anomaly Detection

Anomalies can be:

- **Semantic:** a message whose meaning is unusual.
- **Structural:** a rare event type, or an unusual combination of level, component and parameters.
- **Temporal:** a burst, a repeat, or a rare order of events.

Most existing methods target only one of these.

1.1.3 Unsupervised Learning for Logs

Supervised methods need labelled failures, which are expensive and cannot cover failures that have not happened yet. Unsupervised methods learn what normal looks like and flag what does not fit. Here, the first 60% of each log (the learning window) teaches the detectors normal behaviour. No detector is trained on labels. In the research pipeline (v1), labels are used for evaluation and were also used to choose the view-quality multipliers and λ ; the application (v2) never reads labels.

1.1.4 Multi-View Representation of Log Events

Every line is represented in three views (what it means, what shape it has, and when and in what order it occurs). Each view has its own detector. The views are combined only at the score level, and each view's weight is decided per line from its reliability.

1.1.5 Explainability and Root-Cause Localization

A flag alone does not tell an engineer what to do. LogSight attaches evidence to each flagged line: each view's contribution, the nearest normal example, and the preceding events. It also groups flags into incidents and ranks likely origin components. Root causes are presented as ranked candidates from correlation, not proven causes, because BGL provides no service-dependency information.

1.2 Objectives

1. **Detect anomalies without labels**, learning normal behaviour from the learning window alone.
2. **Analyse each line from three views**, each with a suitable unsupervised detector.
3. **Combine views by reliability** rather than fixed averaging, so one weak view cannot drag down a strong one.
4. **Adapt the threshold** using a rolling, outlier-robust rule, and grade flags by severity.
5. **Explain every decision** with per-view contributions, the nearest normal example and preceding events.
6. **Localize probable root causes** by grouping flags into incidents and ranking components.
7. **Monitor drift** between the learning window and later data.
8. **Evaluate rigorously** with precision, recall, F1, AUC-ROC and AUC-PR on a chronological split, plus an ablation study and an unseen-template test.
9. **Deliver a usable application** with a REST API, persistent storage and an interactive dashboard.

1.3 Existing System

- **Rule and keyword monitoring** catches only anticipated failures, and fixed limits cause false alarms when load changes.
- **Statistical methods on event counts** (PCA, invariant mining) ignore message meaning and fine timing, and flag windows rather than lines.
- **Deep sequence models** (LSTM, transformers) often need large training sequences or labels, rely on one view, and explain little per line.

- **Separate root-cause tools** usually need dependency graphs or tracing, which many systems, including BGL, lack.

Common drawbacks: a single view, fixed or no weighting, fixed thresholds, little explanation, a need for labels or dependency graphs, and detection disconnected from root-cause localization.

1.4 Proposed System

LogSight is an end-to-end, unsupervised, batch-processing system:

1. Ingest and clean the raw BGL log, sorted by microsecond timestamp.
2. Parse each message into a template with Drain3.
3. Split chronologically; the first 60% is the learning window.
4. Build semantic, structural and temporal views for every line.
5. Score each view: prototype distance (semantic), Isolation Forest + LOF (structural), HMM + rolling z-score (temporal).
6. Fuse scores with per-line reliability weights, adjusted by each view's quality on the learning window.
7. Flag lines above a rolling median + MAD threshold and grade them by severity.
8. Monitor drift with a KS test against the learning window, including a shuffle control.
9. Explain the most severe flags with evidence packages.
10. Cluster flags into incidents and rank candidate root-cause components.
11. Store everything in SQLite and serve it through FastAPI to a React dashboard.

Advantages: no labels needed, three complementary views, adaptive weights and thresholds, explainable flags, root-cause candidates without dependency graphs, and a complete working application.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview

Log analysis has moved from manual inspection and hand-written rules to automated parsing, statistical detection, and neural models. Early work built log parsers and datasets and showed that logs carry reliable evidence of failures [1]–[6]. Later work introduced sequence models, semantic embeddings and transformers [7]–[12]. This chapter reviews the work relevant to LogSight under six headings: parsing and datasets, statistical and sequence-based detection, semantic and transformer-based methods, outlier detection and sequence likelihood, robust thresholding and drift, and multi-system detection. It then names the base paper and ends with the research gap.

2.2 Literature Survey

2.2.1 Log Parsing and Datasets

He et al. (2017), Drain [1]. Drain is an online parser that routes each message through a fixed-depth tree, by length and leading tokens, and assigns it to the most similar log group. This turns variable messages into event templates efficiently. LogSight uses Drain3, the maintained implementation. Because the parser is stateful and templates generalise as more messages arrive, LogSight feeds messages in chronological order.

He et al. (2020), Loghub [2]. Loghub is a collection of public system-log datasets with benchmarks for parsing and anomaly detection. It includes the BlueGene/L (BGL) log, the primary dataset of this project.

Oliner and Stearley (2007) [3]. The authors analysed logs from five supercomputers, including BlueGene/L. They showed that such logs are bursty and non-uniform, and introduced the alert tagging that became the BGL labels. This motivates LogSight's temporal features.

2.2.2 Statistical and Sequence-Based Detection

Xu et al. (2009) [4]. The authors built message-count vectors from parsed console logs and applied PCA to detect abnormal executions. Count vectors work per session and ignore message meaning and local timing. LogSight keeps structural features but scores individual lines and adds semantic and temporal views.

Lou et al. (2010) [5]. The authors mined linear invariants among event counts that hold in normal execution and treated violations as anomalies. The invariants are interpretable. LogSight provides interpretability through per-view evidence on each flagged line.

He et al. (2016) [6]. This study compared three supervised and three unsupervised methods. Supervised methods performed better when labels existed, but labels are rarely available in practice. This supports LogSight's unsupervised design and its use of threshold-independent metrics.

Du et al. (2017), DeepLog [7]. DeepLog trains an LSTM to predict the next template key and flags a line when the observed key is not among the predicted ones. It shows that event order carries useful information, but it uses a single view. LogSight models order with an HMM as one of three views.

2.2.3 Semantic and Transformer-Based Methods

Meng et al. (2019), LogAnomaly [8]. LogAnomaly represents templates with semantic vectors, so unseen templates can be matched to similar known ones, and detects sequential and quantitative anomalies without labels. LogSight applies the same idea with Sentence-BERT embeddings.

Zhang et al. (2019), LogRobust [9]. LogRobust uses semantic vectors and an attention-based Bi-LSTM to stay robust when log statements change as software evolves. It is supervised. LogSight targets similar robustness without labels and tests it with an unseen-template evaluation.

Reimers and Gurevych (2019), Sentence-BERT [10]. Sentence-BERT fine-tunes BERT in siamese networks so that sentence embeddings can be compared by cosine similarity. LogSight uses the all-MiniLM-L6-v2 model (384 dimensions) for its semantic view.

Guo et al. (2021), LogBERT [11]. LogBERT trains BERT on log-key sequences with masked key prediction and hypersphere volume minimisation. It captures rich context but needs heavy training and gives little per-line explanation. LogSight pairs a pre-trained embedding model with lightweight classical detectors.

Gao et al. (2021), SimCSE [12]. SimCSE improves sentence embeddings through contrastive learning. It is not in the current pipeline, but fine-tuning embeddings on log messages is a future direction.

Hu et al. (2023) [13] and Zhang et al. (2022), LogST [14]. These works apply Sentence-BERT embeddings to log anomaly detection, confirming that such embeddings suit log messages. LogSight uses the embedding as one of three views rather than the sole signal.

2.2.4 Outlier Detection and Sequence Likelihood

Liu et al. (2008), Isolation Forest [15]. Anomalies are isolated in fewer random partitions than normal points, so path length gives a fast global anomaly score that scales to large data without labels.

Breunig et al. (2000), LOF [16]. LOF compares the local density of a point with that of its neighbours, so it finds points that are outliers relative to their own neighbourhood. The two detectors are complementary: Isolation Forest measures global isolation and LOF measures local density. Together they form LogSight's structural view.

Rabiner (1989), HMM [17]. Rabiner gave the standard treatment of Hidden Markov Models, including the forward algorithm for sequence likelihood. LogSight applies it to windows of template IDs. A low likelihood means the order of events is unusual, which the structural detectors cannot see because each line may look normal on its own.

2.2.5 Robust Thresholding and Drift

Leys et al. (2013) [18]. The authors showed that the mean and standard deviation are distorted by the outliers they are meant to find, and recommended the median and median absolute deviation (MAD). LogSight's threshold is:

$$T = \text{median}(S) + \lambda \cdot 1.4826 \cdot \text{MAD}(S)$$

where S is the fused score in a rolling window of 200 lines, λ sets the sensitivity, and 1.4826 scales MAD to match the standard deviation under normality.

Gama et al. (2014) [19] and Massey (1951) [20]. Gama et al. survey how data distributions change over time and how systems adapt. The two-sample Kolmogorov-Smirnov (KS) test compares two distributions without assuming their form. LogSight applies it to each line's embedding distance from the learning-window centroid and to template frequency, comparing later windows with the learning window. It adds a shuffle control, splitting the learning window randomly and testing the halves against each other, so that temporal clustering in bursty logs is not mistaken for real drift.

2.2.6 Multi-System Detection

Zang et al. (2024), MLAD [21]. MLAD proposes a unified model for several log sources. LogSight is evaluated on BGL only, but its template-based, embedding-based design could be applied to other datasets.

2.3 Base Paper

The base paper for this work is LogAnomaly by Meng et al. (2019) [8]. It detects sequential and quantitative anomalies in unstructured logs without labels, and it represents templates with semantic vectors so that unseen templates can be matched to similar known ones. LogSight keeps this unsupervised, semantic and unseen-template-aware setting and extends it in five ways:

1. It uses Sentence-BERT embeddings of log messages, scored against KMeans prototypes of normal messages.

2. It adds a structural view and a temporal view, so three views are scored instead of one.
3. It fuses the three scores with per-line reliability weights.
4. It replaces fixed thresholds with a rolling median + MAD threshold.
5. It adds per-line evidence, incident grouping, root-cause ranking and drift monitoring.

2.4 Research Gap

Relative to the base paper [8] and the other reviewed work, five gaps remain.

1. **Single-view detection.** Count-based methods [4], [5], sequence models [7] and semantic methods [8], [10] each capture one aspect of abnormal behavior and miss anomalies visible only from another angle.
2. **Fixed or uncalibrated combination.** Structural, semantic and sequence scores differ in scale and reliability, so a weak signal can dominate. LogSight weights each view per line by its reliability and its informativeness on the unlabeled learning window.
3. **Fixed or non-robust thresholds.** Fixed thresholds fail when normal behavior changes, and mean-based thresholds are distorted by extreme scores. LogSight uses a rolling median + MAD threshold [18].
4. **Detection separate from explanation and root cause.** Root-cause methods usually need dependency graphs or tracing, which BGL lacks. LogSight attaches per-view evidence to each flag, groups flags into incidents, and ranks candidate components by correlation.
5. **Drift tests without controls.** On bursty logs, consecutive chunks can look different even when nothing has changed. LogSight adds a shuffle control for this reason.

LogSight does not replace the algorithms reviewed above. It integrates them into one unsupervised pipeline with three views, reliability-weighted fusion, a robust threshold, evidence-based explanation, root-cause ranking and controlled drift monitoring.

CHAPTER 3

SYSTEM DESIGN

3.1 Dataset Loading

- **Dataset:** the BGL log from Loghub, with 4,747,963 lines covering 3 June 2005 to 4 January 2006.
- **Loading:** each line is split into ten fields on runs of whitespace, because splitting on a single space shifts columns where the file has double spaces. Ten lines with displaced fields are removed, leaving **4,713,483** lines. Lines are sorted by the microsecond time field, since the Unix timestamp has only second precision and would scramble events within a burst.
- **Labels:** kept but never used to train any detector, and never read by the application. Only the research evaluation scripts read them; in v1 their results were also used to choose the quality multipliers and λ .
- **Demo samples:** five continuous 50,000-line samples cut from different parts of the log. Each sample is the stretch within its part of the log whose labelled share is closest to the full file's 7.39%, so the actual shares range from 2.4% to 7.5%. Lines are never shuffled.
- **Storage:** intermediate research outputs are saved as Parquet and NumPy arrays.

3.2 Development Environment

Hardware: All development and experiments were run on a Windows laptop with an AMD Ryzen 7 250 processor, 16 GB RAM, a 954 GB drive and an NVIDIA GeForce RTX 4060 Laptop GPU. The full run needs several GB of RAM because one 384-value embedding is held per line. A GPU is optional; Sentence-BERT uses CUDA if available. Development was on Windows, with Linux containers for Docker.

Software:

- **Pipeline and backend:** Python 3.10, pandas, NumPy, SciPy, PyArrow.
- **Parsing:** Drain3 0.9.11.
- **Semantic model:** sentence-transformers (all-MiniLM-L6-v2), PyTorch.
- **Detectors:** scikit-learn 1.7.2 (MiniBatchKMeans, IsolationForest, LocalOutlierFactor), hmmlearn 0.3.3.
- **Backend and storage:** FastAPI, Uvicorn, Pydantic, SQLite through SQLAlchemy 2.0 (portable to PostgreSQL).
- **Testing:** pytest.
- **Frontend:** React 19, TypeScript, Vite, React Router, GSAP, Motion (Node 22).
- **Deployment:** Docker, Docker Compose, nginx.
- **IDE:** Visual Studio Code.

3.3 Architecture

LogSight has five layers:

1. **Ingestion and parsing:** clean, sort, mine templates with Drain3.
2. **Multi-view detection:** semantic, structural and temporal views, each outputting an anomaly score.
3. **Decision:** reliability-weighted fusion, adaptive threshold, then severity (Low, Medium, High, Critical).
4. **Explanation and localization:** evidence packages, incidents with ranked components, and KS-based drift monitoring.
5. **Storage and presentation:** SQLite, then the FastAPI REST API, then the React dashboard.

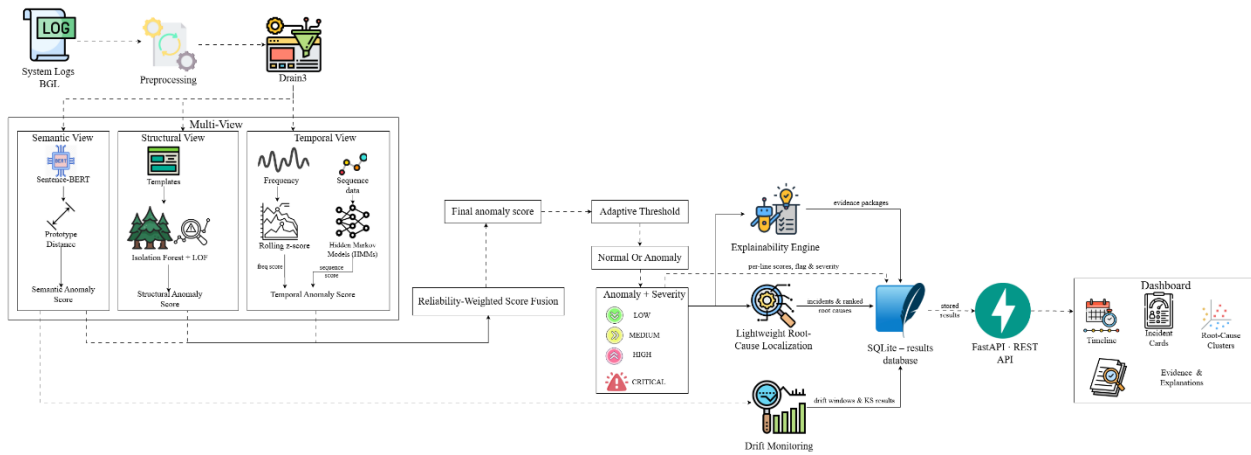


Fig 3.1: System architecture of LogSight

Code organisation:

- engine/ holds the pipeline functions (ingest, parse, features, detection, rca, evidence).
- backend/app/pipeline/experiment.py is the only module that imports the engine, and stages.py calls the engine functions in order.
- backend/config/pipeline.yaml holds every parameter. A null value inherits the engine default, so the engine stays the single source of truth.

3.4 Multi-View Detection Model

Semantic view

- Input: the text of each line.
- Model: Sentence-BERT all-MiniLM-L6-v2 (384 dimensions).
- Detector: MiniBatchKMeans with 30 prototypes fitted on learning-window embeddings.
- Score: $1 - \text{maximum cosine similarity to any prototype}$.

Structural view

- Input: template ID, parameter count, level rank, component, type, and template frequency counted on the learning window only.
- Detectors: Isolation Forest (100 trees) and LOF (20 neighbours, novelty mode, fitted on a 30,000-line learning-window sample).
- Score: mean of the two min–max scaled outlier scores.

Temporal view

- Input: the template-ID sequence and seven timing features.
- Detectors: a 10-state categorical HMM fitted on the learning-window sequence (20-line windows scored every 10 lines), and a rolling z-score (window 50) of the 60-second event frequency.
- Score: mean of the scaled sequence score and the scaled $|z|$.

3.5 Fusion, Threshold and Severity

- **Reliability per view:**
 - Semantic: mean cosine similarity to the 10 nearest prototypes.
 - Structural: log-scaled, min–max normalised template frequency.
 - Temporal: inverse of the rolling variance of event frequency (window 100).
- **Weights:** a row-wise softmax over the three reliabilities, multiplied by each view's quality and re-normalised. In the application (v2), quality is the spread between the 99th and 50th percentiles of the view's scores on the learning window, scaled so the best view is 1, with a floor of 0.1. No labels are read. The research pipeline (v1) used fixed multipliers instead (Section 4.4).
- **Final score:** the weighted sum of the three view scores.
- **Threshold:** rolling median + $\lambda \times 1.4826 \times$ rolling MAD, with window 200 and $\lambda = 1.5$.
- **Severity (flagged lines only):** $0.4 \times \text{final score} + 0.25 \times \text{persistence} + 0.2 \times \text{normalised frequency} + 0.15 \times \text{template rarity}$. Buckets are quantiles among flagged lines: bottom 50% Low, 50–85% Medium, 85–98% High, top 2% Critical.

3.6 Explainability, Drift and Root-Cause Module

- **Evidence package** (up to 200 most severe flags per run): the line, component, node and template; final score and severity; each view's score and weight; the dominant view (largest score $\times$ weight); the nearest normal line of the same template by cosine similarity; and the five preceding events.
- **Drift monitoring:** two signals (embedding distance from the learning-window centroid, and template frequency). Each is compared with a 50,000-value learning-window sample by a two-sample KS test over 50,000-line windows. Drift is

flagged after three consecutive windows with $p < 0.01$. A shuffle control splits the learning window randomly and tests the halves against each other.

- **Root-cause localization:**

- Flags within 10 minutes of each other are chained into one incident.
- Component pairs appearing in the same incident are counted (co-occurrence).
- Each incident's template counts and components form its signature.
- Components are ranked by $0.35 \times \text{first-occurrence priority} + 0.30 \times \text{average severity} + 0.20 \times \text{in-incident frequency} + 0.15 \times \text{co-occurrence centrality}$, each min–max normalised within the incident.

3.7 Backend and Database Module

- **Background runs:** uploads are processed by a thread-pool worker. A run moves through queued → running → completed or failed. The current stage and a timed stage log are stored for the progress display. Runs interrupted by a restart are marked failed at start-up, and each run stores its parameter snapshot.
- **Database (9 tables):** pipeline_runs, log_events, templates, incidents, root_cause_candidates, component_cooccurrence, drift_summaries, drift_windows, evidence_packages. All link to pipeline_runs through run_id.
- **REST API (22 endpoints under /api/v1):**
 - Health and config: GET /health, GET /config/pipeline.
 - Run management: POST /runs (returns 202), GET /runs, GET /runs/{id}, DELETE /runs/{id}.
 - Overview data: summary, timeline, score-timeline, baseline, components.
 - Flagged lines and evidence: anomalies, anomaly-kinds, events/{row}, evidence, templates.
 - Root cause: incidents, incidents/{id}, root-causes, root-cause-clusters, cooccurrence.
 - Drift: drift.

3.8 Visualization Module

The dashboard is a React single-page application. Every number comes from the API, and only explanatory wording lives in the frontend. Pages:

- **Landing, Upload, run receipt (live stage progress), Runs.**
- **Overview:** headline counts, anomaly timeline, severity ring.
- **When and where:** timeline by day or hour, components by anomaly count.
- **Incidents:** 2D root-cause cluster map, incident list, and a "How this root cause was chosen" detail view.

- **Three views:** weight and flags driven per view, detector facts, parameters, drift check.
- **Flagged lines:** grouped by kind, with line detail showing evidence and Shapley attribution.
- **Templates, Printable summary** (print or JSON download).

CHAPTER 4

METHODOLOGY

This work has two implementations that share the same detection idea but serve different purposes. The research pipeline (v1) measures detection quality on the full dataset. The application (v2) runs on uploaded logs and explains its results. Table 4.1 compares them.

	Research pipeline (v1)	Application (v2)
Purpose	Measure detection quality	Run on uploaded logs and explain results
Data	Full BGL, 4,713,483 lines	50,000-line BGL samples
Semantic embedding	One vector per template	One vector per distinct line
View quality q	Fixed 0.2 / 1.0 / 0.5, chosen from test AUC	Measured without labels (P99 – P50 on the learning window)
Severity buckets	Fixed cut-offs (0.3, 0.6) + top 2% Critical	Quantiles 50 / 85 / 98%
Labels	Used for evaluation, and to choose q and λ	Never used
Reported numbers	AUC-ROC 0.7237, AUC-PR 0.1900, ablation, unseen templates	Table 5.7, flag counts, severity split

Table 4.1 Research pipeline (v1) versus application (v2)

All accuracy figures describe v1. v2 has no labelled evaluation; its results show behaviour, not accuracy.

4.1 Data Collection and Preprocessing

1. **Load:** each raw line is split with split (None, 9) into ten fields. Lines with fewer than ten fields are skipped.
2. **Type conversion:** timestamp becomes numeric, and time is parsed with %Y-%m-%d-%H.%M.%S.%f.
3. **Cleaning:** rows with missing timestamp, time or content are dropped, along with the ten rows whose level field held stray values. Text fields are trimmed.
4. **Ordering:** rows are sorted by time and re-indexed. Row order is the reference every later stage stays aligned with.

Result: 4,713,493 $\rightarrow$ 4,713,483 lines.

4.2 Log Parsing and Exploratory Analysis

- Labelled anomaly rate: **7.39%** (348,460 lines).
- Dominant level: INFO (3,701,880 lines), followed by FATAL (854,658), ERROR (112,355), WARNING (23,357), SEVERE (19,213), FAILURE (1,714) and Kill (306).
- Inter-arrival time: median 0.0265 s, 99th percentile 2.108 s. This confirmed that BGL is bursty and motivated the burst and repeated-event features.

Parsing. Drain3 (default configuration) mines templates from the content field in chronological order. It is stateful: the first message of a kind becomes the template verbatim, and later similar messages generalise it with `<*>` wildcards.

Results on the full log:

- 1,819 templates.
- 1,161 singletons (63.83%).
- The most frequent template covers 1,706,751 lines.

The singletons are mostly register-dump lines whose hexadecimal values share no structure. Retuning Drain3 to merge them risked merging genuinely different events, so the template count was kept and the semantic and temporal views cover the gap.

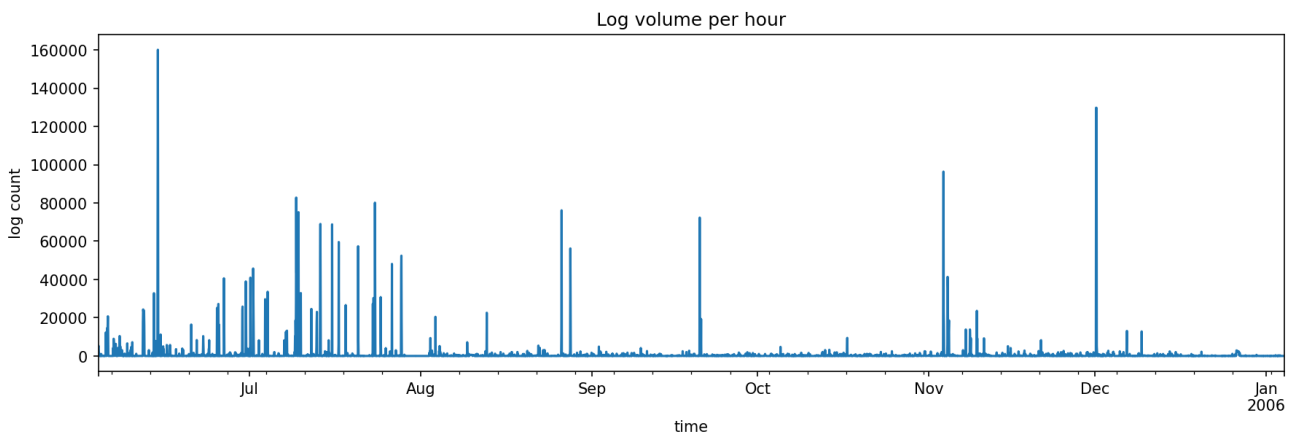


Fig 4.1: Hourly log volume of BGL

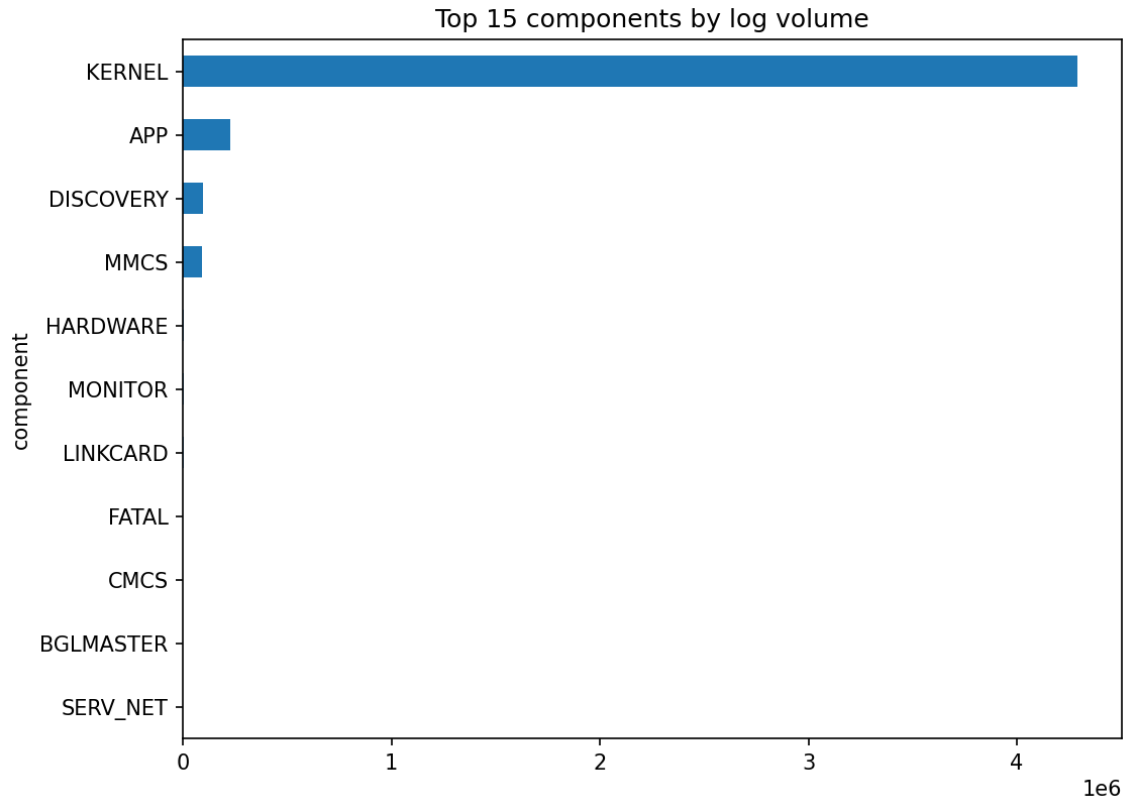


Fig 4.2: Top 15 components by log volume

4.3 Multi-View Feature Engineering

Chronological split (60/20/20): 2,828,089 / 942,697 / 942,697 lines. The split is never random, because a random split would let detectors learn from the future. The learning window is not cleaned of anomalies, since that would need labels, so detectors are expected to tolerate small contamination.

Semantic features. Each distinct line text is embedded once with all-MiniLM-L6-v2 (batch size 256), and the vectors are broadcast back to all lines with that text.

- **Research (v1):** one vector per template ID, using the last, most generalized template text.
- **Application (v2):** one vector per distinct line text, so two lines of one template can differ. Above 200,000 distinct texts it falls back to one vector per template.

Structural features:

- **template_id:** Drain3 cluster ID.
- **param_count:** number of extracted parameters.
- **level_rank:** INFO 0, WARNING 1, SEVERE 2, ERROR 3, FAILURE 4, FATAL/Kill 5, unknown -1.
- **component, type:** one-hot encoded.

- `template_global_freq`: occurrences of the template in the learning window only (0 if unseen). Counting it over the whole log was a leakage bug found during evaluation.

Temporal features (past data only, no look-ahead):

- `time_since_prev`: seconds since the previous line.
- `rolling_freq`: lines in the last 60 seconds.
- `burst_rate`: of the last 5 lines, how many fall within 1 second.
- `template_novelty`: 1 if the template is seen for the first time.
- `transition_prob`: causal empirical probability of this template after the previous one.
- `rolling_entropy`: Shannon entropy of templates in the last 50 lines.
- `repeated_event_ratio`: share of the last 10 lines with the same template.

4.4 Anomaly Scoring and Reliability-Weighted Fusion

Notation: mm is min–max scaling, c_k are the KMeans prototypes, e_i is the embedding of line i , a_i and b_i are the Isolation Forest and LOF outlier scores (negated decision functions), h_i is the HMM negative log-likelihood per line of the 20-line window ending at i (computed every 10 lines and forward-filled), and z_i is the rolling z-score of `rolling_freq`.

- **Semantic:** $s_{\text{sem}}(i) = 1 - \max_k \cos(e_i, c_k)$, with 30 clusters, batch 10,000, $n_{\text{init}} 10$, seed 42.
- **Structural:** $s_{\text{str}}(i) = \frac{1}{2} [mm(a_i) + mm(b_i)]$.
- **Temporal:** $s_{\text{tem}}(i) = \frac{1}{2} [mm(h_i) + mm(|z_i|)]$.

In the application, a window containing a template the HMM never saw in training gets the highest finite score seen so far, instead of reusing the previous score.

Fusion. Let $r_{i,v}$ be the reliability of view v on line i , and q_v the quality of view v . The weight of view v on line i and the fused score are:

$$w_{i,v} = \frac{q_v e^{r_{i,v}}}{\sum_u q_u e^{r_{i,u}}}$$

$$S_i = \sum_v w_{i,v} s_v(i)$$

This is a softmax over the reliabilities in which each view is scaled by its quality (equivalently, $\ln q_v$ is added to $r_{i,v}$), so the three weights on every line sum to 1.

- **Research (v1):** fixed quality multipliers of 0.2 / 1.0 / 0.5 (semantic, structural, temporal), set after per-view AUC was measured on the labelled test split.

- **Application (v2):** $q_v = \max\left(0.1, \frac{P_{99}(s_v) - P_{50}(s_v)}{\max_u[P_{99}(s_u) - P_{50}(s_u)]}\right)$, where P_{99} and P_{50} are the 99th and 50th percentiles of a view's scores on the learning window. This uses no labels.

4.5 Adaptive Thresholding, Severity and Drift

Threshold: $\tau_i = \text{median}_{200}(S) + \lambda \cdot 1.4826 \cdot \text{MAD}_{200}(S)$, with $\lambda = 1.5$. Line i is flagged if $S_i > \tau_i$. The median and MAD barely move when a few anomalies fall in the window, unlike the mean and standard deviation.

Severity (flagged lines only): $\text{sev}_i = 0.4 S_i + 0.25 \text{persistence}_i + 0.2 \text{freq}_i^{\text{norm}} + 0.15 \text{rarity}_i$.

- **v1:** fixed cut-offs (0.3, 0.6), with the top 2% marked Critical; under these cut-offs, no line fell in High.
- **v2:** quantile buckets (50 / 85 / 98%).

Default parameters:

- **Split:** 0.6 / 0.2.
- **Semantic features:** all-MiniLM-L6-v2, batch 256, per line, up to 200,000 distinct lines.
- **Temporal features:** 60 s frequency window.
- **Semantic scoring:** 30 prototypes.
- **Structural scoring:** 100 trees, contamination 0.1, LOF 20 neighbours, sample 30,000.
- **Temporal scoring:** 10 HMM states, 50 iterations, window 20, stride 10, z-score window 50.
- **Reliability:** semantic $k = 10$, temporal window 100, minimum quality 0.1.
- **Threshold:** window 200, $\lambda = 1.5$.
- **Drift:** window and step 50,000, $p = 0.01$, 3 consecutive windows, reference sample 50,000, seed 42.
- **Incidents:** 10 min gap.
- **Root cause:** 0.35 / 0.30 / 0.20 / 0.15.
- **Evidence:** 200 packages per run.

Drift. Learning-window values are sub-sampled to 50,000 for each signal, and each 50,000-line window is compared with that sample using a two-sample KS test. Without sub-sampling, the test's power on 2.8 million reference values made trivial differences look significant. The shuffle control randomly splits the learning window, and a near-zero statistic confirms the computation is correct.

4.6 Evidence Generation and Root-Cause Localization

Evidence. Flags are ordered by severity and a package is built for the top 200. The nearest normal example is searched among normal lines of the same template (falling back to all normal lines), so it answers "what does a normal instance of this event look like". The dominant view is chosen by score $\times$ weight, the same arithmetic the fusion used.

Attribution. Because the fused score is a weighted sum, each view's contribution is exactly $\varphi_v = w_v (s_v - \bar{s}_v)$, where $\bar{s}_v$ is the mean of s_v over non-flagged lines. With the line's weights held fixed, the base value $B = \sum_v w_v \bar{s}_v$ plus the three contributions equals the final score, $S = B + \varphi_{\text{sem}} + \varphi_{\text{str}} + \varphi_{\text{tem}}$. The dashboard shows this as a Shapley-style waterfall.

Root cause.

- **Incidents:** flags sorted by time, with a new incident when the gap exceeds 10 minutes. Gaps chain, so a long cascade stays one incident.
- **Co-occurrence:** unordered component pairs counted per incident. This is correlation, not causation.
- **Signature:** per incident, the count of each template, the components involved, and start and end times.
- **Ranking factors per component:** first-occurrence priority $1/(1 + \text{seconds since incident start})$, average severity, in-incident frequency, and co-occurrence centrality. Each is min-max normalised within the incident, then combined with weights 0.35 / 0.30 / 0.20 / 0.15.

4.7 Application Integration

- **Upload:** POST /runs streams the file to disk in 1 MB chunks, enforces the upload limit (1,024 MB by default), rejects empty files and returns 202 immediately.
- **Execution:** the worker loads pipeline.yaml, records the effective parameters, runs the ten stages while reporting each, then persists results in chunks of 50,000 rows.
- **Detector facts:** for every run the backend records what each fitted detector actually was (number of prototypes, trees, neighbours, HMM states and iterations, and lines each detector learned from). The test suite checks that every detector was fitted on that upload's learning window.
- **Import mode:** python -m app.cli import-artifacts loads the full-dataset research outputs and recomputes only root-cause analysis, keeping it consistent with the saved flags.
- **Deployment:** docker compose up --build builds the backend and the frontend (served by nginx, which proxies /api/). The app runs on port 8080.

4.8 Evaluation Protocol

- Labels (label != "-" means anomaly) are read only in research/evaluation/.
- Metrics are computed on the test split (rows from 3,770,786, 942,697 lines): precision, recall and F1 on the flags, and AUC-ROC and AUC-PR on the final score.
- Accuracy is not used. With 7.39% anomalies, flagging nothing would already give about 93%.
- **Ablation:** each configuration (semantic only, structural only, temporal only, semantic + structural, all three) averages its views with equal weights and uses the same threshold.
- **Unseen-template test:** test lines whose template never occurs in the learning window are evaluated separately from lines with seen templates.
- **Threshold sweep:** $\lambda \in \{1.0, 1.5, 2.0, 2.5, 3.0, 4.0\}$ on the test split, reporting F1 and flagged volume. Table 5.6 shows the two values compared for the final choice.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Overview

Two sets of results are presented: the research evaluation (v1) on the full BGL log, and the application (v2) on 50,000-line BGL samples. v2 uses per-line embeddings, label-free view quality, quantile severity buckets and a revised HMM novelty rule. The research numbers therefore describe v1, and a full-scale evaluation of v2 is future work.

5.2 Functional Verification

The backend's pytest suite collected 28 tests, all passing. It covers:

- **API:** health, summary, timelines, component ranking, filtered and sorted anomalies, event detail, drift, templates, evidence, incidents, root-cause clusters, run deletion, rejection of empty and oversized uploads, and 404 for unknown runs.
- **Configuration:** defaults inherited from the engine, overrides applied, unknown parameters rejected.
- **End-to-end:** a 4,000-line upload runs every stage in order, every detector is fitted on that upload's learning window, and per-view flag counts add up to the total.
- **Detection units:** an unseen-template HMM window scores highest and stays finite, and severity buckets follow the 50/85/98% quantiles.

The script `verify_detectors.py` also re-derives every score from the fitted detectors' outputs and checks that the fusion weights sum to 1 on every line.

5.3 Dataset and Parsing Results

The dataset and split statistics are given in Table 5.1.

Measure	Value
Raw lines	4,713,493
Cleaned lines	4,713,483
Time range	2005-06-03 15:42:50 to 2006-01-04 08:00:05
Labelled anomalies	348,460 (7.39%)
Templates	1,819
Singleton templates	1,161 (63.83%)
Median template frequency	1
Learning window (train)	2,828,089 lines
Validation	942,697 lines
Test	942,697 lines (46,283 labelled anomalies, about 4.9%)

Table 5.1 Dataset, parsing and split statistics

5.4 Detection Performance Analysis

The fused score was improved in three steps, summarised in Table 5.2.

Step 1: a score worse than chance. The first fused score gave an AUC-ROC of 0.456 on the test split. The per-view values were 0.306 (semantic), 0.547 (structural) and 0.428 (temporal). The semantic view was anti-correlated with anomalies. With one embedding per template, every line of a template has an identical vector, so an anomalous instance of a common template cannot be told apart from its normal instances.

Step 2: view quality multipliers. Applying multipliers of 0.2 / 1.0 / 0.5 after the softmax raised the fused AUC-ROC to 0.554.

Step 3: leakage fix. `template_global_freq` had been counted over the whole log, including the test period. Counting it on the learning window only raised the fused AUC-ROC to 0.7237. The fix changed only the structural view, whose AUC-ROC rose from 0.547 to 0.830. The semantic (0.306) and temporal (0.428) values were unchanged.

Stage	Semantic	Structural	Temporal	Fused
Initial fused score	0.306	0.547	0.428	0.456
+ view quality multipliers	0.306	0.547	0.428	0.554
+ template-frequency leakage fix (final v1)	0.306	0.830	0.428	0.7237

Table 5.2 AUC-ROC progression across corrections

The final v1 test metrics are given in Table 5.3.

Metric	Value
Precision	0.0806
Recall	0.1625
F1	0.1078
AUC-ROC	0.7237
AUC-PR	0.1900
Test lines	942,697
True anomalies	46,283
Flagged lines	93,244

Table 5.3 Final research test metrics (v1, $\lambda = 1.5$)

A random score would achieve an AUC-PR equal to the test anomaly rate of about 4.9%. The AUC-PR of 0.1900 is therefore about 3.9 times the random baseline.

5.5 Ablation and Unseen-Template Results

Ablation

Each configuration in Table 5.4 averages its views with equal weights and uses the same threshold.

Configuration	AUC-ROC	AUC-PR
Semantic only	0.306	0.063
Structural only	0.830	0.657
Temporal only	0.428	0.052
Semantic + structural	0.465	0.067
All three (equal weights)	0.470	0.048

Table 5.4 Ablation results

The ablation shows that:

- Averaging all three views equally (0.470) is far worse than the structural view alone (0.830), because the two weak views drag a strong one down.
- The weighted v1 pipeline (0.7237, with quality multipliers) recovers most of that gap, which shows that weak views must be down-weighted.
- Structural-only still beats the tuned v1 pipeline. This motivated measuring view quality from the data in v2, and it remains an open point.

Unseen versus seen templates

Subset	Lines	True anomalies	Base rate	Precision	AUC-ROC	AUC-PR	AUC-PR ÷ base rate
Unseen templates	749,626	43,679	5.83%	0.093	0.658	0.252	4.3×
Seen templates	193,071	2,604	1.35%	0.029	0.936	0.103	7.7×
All test lines	942,697	46,283	4.91%	0.081	0.724	0.190	3.9×

Table 5.5 Unseen vs seen template results (test split)

Three points stand out:

- 79.5% of test lines carry templates never seen in the learning window, so unseen patterns are the common case, not an edge case.
- Seen templates rank anomalies very well, but precision is low (0.029) because anomalies are rare among them.
- Unseen templates have the higher raw AUC-PR mainly because anomalies are about four times more common there (5.83% vs 1.35%). Relative to the base rate, seen templates do better (7.7× vs 4.3×), which matches their higher AUC-ROC.

5.6 Threshold and Drift Results

Threshold sensitivity. λ was swept over $\{1.0, 1.5, 2.0, 2.5, 3.0, 4.0\}$ on the test split (Table 5.6 shows the two candidates). $\lambda = 1.0$ gave the highest F1 but flagged about 27% of the test set. $\lambda = 1.5$ kept almost the same F1 while flagging about 10%, which is more practical for human review.

λ	F1	Flagged lines	Share of test set
1.0	0.1117	253,001	about 27%
1.5	0.1078	93,244	about 10%

Table 5.6 Threshold comparison on the test split

λ and the $v1$ multipliers were chosen by looking at test-split results, and the leakage bug was also found through test AUC. The 20% validation split (rows 2,828,089 to 3,770,785) was created but not used. Tuning on it instead is listed as future work.

Drift. Sequential windows flagged drift at row 100,000, which lies inside the learning window itself. The shuffle control on the same data showed no difference ($KS \approx 0.001$, $p > 0.3$ for both signals). The computation is therefore correct, and BGL's dominant templates simply occur in time-concentrated bursts, which makes any two consecutive chunks look different. On BGL, the sequential flags should be read as a demonstration of the method. In the application, runs shorter than several 50,000-line windows are reported as too short to judge drift.

5.7 Application Results

Effect of semantic view $v2$

The effect was measured on BGL sample 1 (50,000 lines, 16–21 June 2005), as shown in Table 5.7.

Measure	$v1$	$v2$
Mean weight (semantic / structural / temporal)	11.8% / 69.3% / 19.0%	37.7% / 40.2% / 22.1%
Flags driven (semantic / structural / temporal)	0 / 3,294 / 4,485	3,115 / 1,676 / 4,207
Distinct semantic scores	192	21,111

Table 5.7 Semantic view $v1$ vs $v2$ (BGL sample 1)

The semantic view went from driving no flags to driving 3,115, and the weights are now balanced. Embedding about 21,000 distinct lines takes roughly 40 seconds on CPU. Total flags rose by about 16% (from 7,779 to 8,998). This shows the view is active, but not that it is more accurate, because the application reads no labels.

Run summary (sample 1)

- 50,000 readable lines, 194 templates, 8,998 flagged lines (18.0%) and 96 incidents.

- Severity: Critical 180 (2%), High 1,170 (13%), Medium 3,149 (35%), Low 4,499 (50%).

Why the flag rate exceeds the labelled rate. The flag rate (18.0%) is above the labelled rate (6.2%) because the rolling MAD threshold always flags the upper tail of each window. On synthetic noise with the same window (200) and λ (1.5), it flagged 0.4% of uniform noise, 6.9% of normal noise and 19.7% of log-normal noise. Part of the flag rate therefore depends on the shape of the score distribution, not only on real anomalies. Severity grading and incident grouping help reviewers focus on the most important flags.

Demo samples

Five 50,000-line samples were used, with labelled-anomaly rates of 6.2%, 2.4%, 3.6%, 7.5% and 7.4%. In sample 2, 202 lines have no message and are skipped as unreadable.

5.8 Dashboard Results

- **Overview** (Figure 5.1): four headline tiles, the anomaly timeline (worst score per slice against the moving cutoff, with the learning window shaded and incident markers), and the severity ring.
- **Incidents** (Figure 5.2): a 2D cluster map in which each circle is a likely-origin component, sized by incidents, with its largest incidents as dots. Each incident has a four-step strip showing how the candidate was chosen, with normalised ranking factors as bars.
- **Three views:** each view's average weight, flags driven, detector facts as fitted, and the drift check.
- **Flagged-line detail** (Figure 5.3): the evidence package and an exact attribution of the final score to the three views.
- **Interface:** motion respects the reduced-motion preference, and the layout works at desktop and mobile widths.

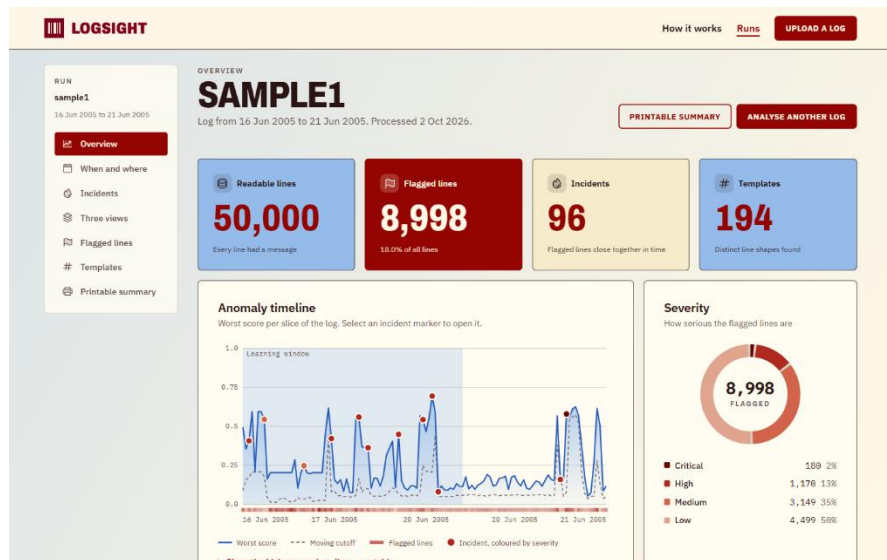


Fig 5.1 Overview dashboard

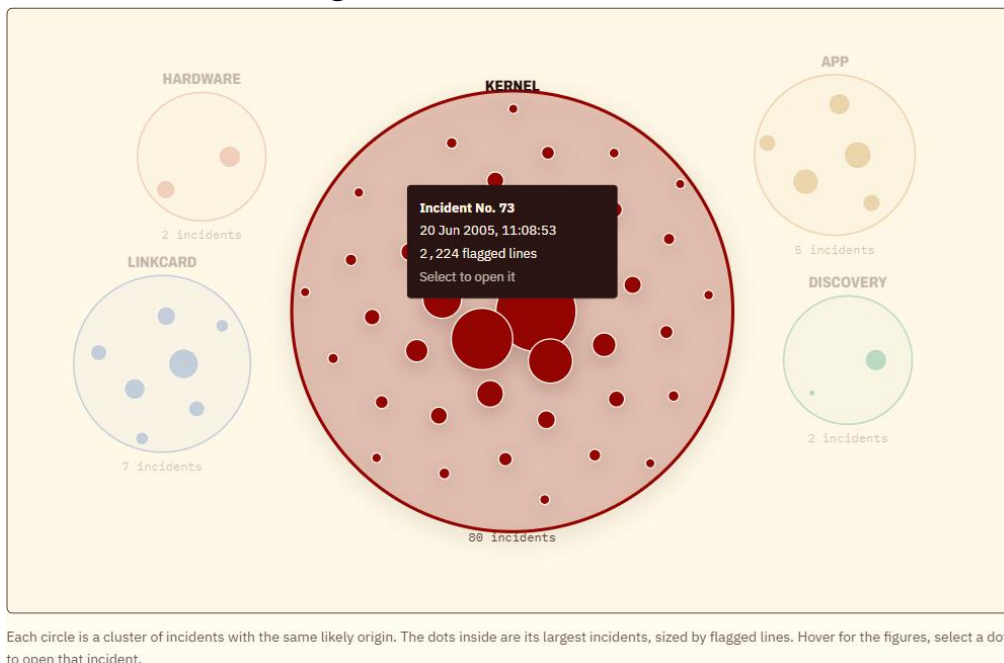


Fig 5.2 Incidents page: 2D root-cause cluster map

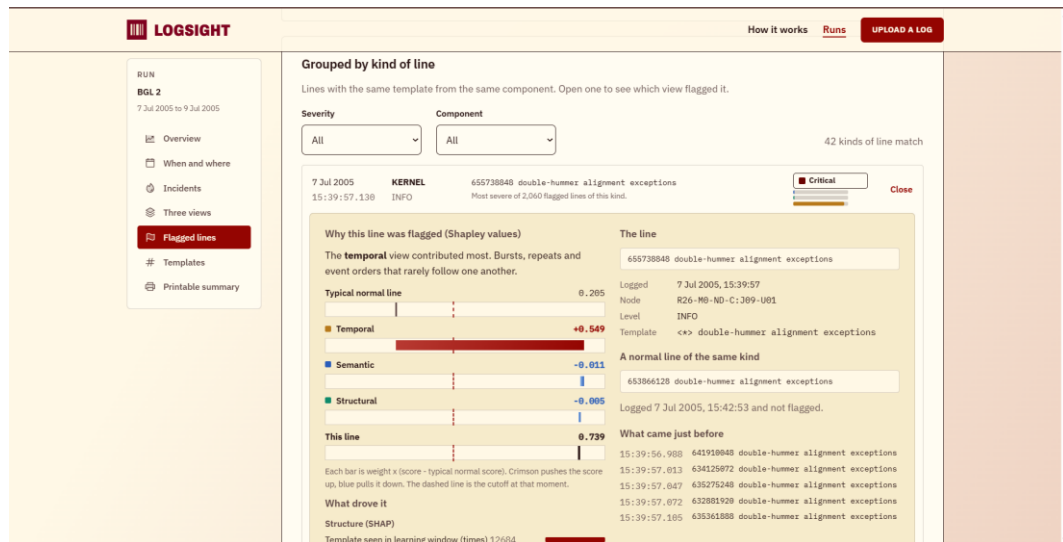


Fig 5.3 Flagged line detail with Shapley attribution

5.9 Comparative Discussion

LogSight is the only approach reviewed that combines all of the capabilities in Table 5.8: no labels needed, semantic, structural and temporal views, per-line reliability weighting, an adaptive median + MAD threshold, per-line evidence, root-cause candidates and drift monitoring with a control. Rule-based methods and PCA/invariants lack most of these. DeepLog, LogRobust and LogBERT cover only the sequence or semantic side, with limited explanation.

Capability	Rule-based	PCA / invariants	DeepLog	LogRobust	LogBERT	LogSight
Works without labels	Yes	Yes	Yes	No	Yes	Yes
Semantic view	No	No	No	Yes	No	Yes
Structural view	Partly	Yes	No	No	No	Yes
Temporal / sequence view	Partly	No	Yes	Yes	Yes	Yes
Per-line reliability weighting	No	No	No	No	No	Yes

Adaptive robust threshold	No	No	No	No	No	Yes
Per-line evidence	Rule text	Limited	Limited	Attention	Limited	Yes
Root-cause candidates	No	No	Workflow diagnosis	No	No	Yes
Drift monitoring	No	No	Model update	No	No	Yes

Table 5.8 Feature comparison with existing approaches

The comparison covers capabilities only. No numeric comparison with published results is made, because the reviewed works use different preprocessing, windowing (session or sliding window rather than per line) and splits.

5.10 Discussion and Insights

Insights

- **Weighting matters more than adding views.** Simply adding views can hurt. The benefit comes from weighting them by reliability and quality.
- **Leakage is easy to introduce and expensive to miss.** One feature computed over the whole log was the largest source of error, and fixing it gave the largest AUC gain.
- **Unseen patterns are the normal case.** Nearly four in five test lines had unseen templates.
- **Semantic design choices have measurable effects.** Per-template embeddings made the semantic view useless within a template, while per-line embeddings made it the second most influential view.
- **Drift tests need controls.** Without the shuffle control, temporal clustering in BGL would have been mistaken for drift.

Limitations

- Only the BGL format is supported, and processing is batch only.
- The full 4.7M-line log takes hours and several GB of memory.
- Research metrics are from v1, and v2 has not been evaluated at full scale.
- λ and the v1 multipliers were tuned on the test split.
- v2 severity buckets are relative, so about 2% of flags are always Critical.
- The first-occurrence priority $1/(1 + \text{seconds})$ decays very fast and can dominate the ranking.
- The 10-minute incident gap can split long cascades.
- Root causes are correlation-based candidates, not proof.

- The 20% validation split was created but not used for tuning.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

This project designed, built and evaluated LogSight, an unsupervised multi-view system for log anomaly detection and root-cause localization. Every log line is analysed for meaning, structure and timing, with a dedicated unsupervised detector for each view. The three view scores are fused by per-line reliability, flagged by a robust adaptive threshold and graded by severity. Each flag is explained with evidence, related flags are grouped into incidents, candidate root causes are ranked, and drift is checked with a KS test and a shuffle control. On the full BGL log (4,713,483 lines), the research pipeline reached an AUC-ROC of 0.7237 and an AUC-PR of 0.1900 on a chronological split, and diagnosing a leakage bug and a misleading view raised the AUC-ROC from 0.456. The ablation showed that equal-weighted fusion (0.470) underperformed the best single view (0.830); weighted fusion (0.7237) recovered most of that gap but did not exceed the structural view alone. The system also remains above chance (AUC-ROC 0.658) even though nearly 80% of test lines carry templates unseen in the learning window. The work was delivered as a full-stack application with a FastAPI backend of 22 endpoints, a nine-table SQLite database, a React dashboard and Docker deployment, covered by 28 automated tests. A redesigned semantic view and label-free view quality made all three views contribute meaningfully, and the application never reads labels.

6.2 Future Enhancements

1. **Full-scale evaluation of v2** with per-line embeddings and label-free view quality.
2. **Validation-based tuning** of λ , prototypes, HMM states and severity weights.
3. **Better root-cause ranking:** log-scale the first-occurrence gap, test the weights, use real blast radius (distinct components and nodes per incident) in severity, and adapt the incident gap.
4. **Better drift reference:** randomly or stratified-sampled reference windows, and smaller windows for short logs.
5. **More datasets:** parsers for HDFS and Thunderbird.
6. **Streaming mode** with periodic retraining triggered by drift.
7. **Stronger semantic model:** fine-tune embeddings on log messages (for example SimCSE-style contrastive learning).
8. **Optional LLM explanation** as a presentation layer only, without affecting detection.
9. **Scalability:** vectorise feature and threshold loops, save fitted models, and move to PostgreSQL.

10. Production features: authentication, schema migrations and multi-user access.

REFERENCES

- [1] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Honolulu, HI, USA, 2017, pp. 33–40.
- [2] S. He, J. Zhu, P. He, and M. R. Lyu, "Loghub: A large collection of system log datasets towards automated log analytics," arXiv:2008.06448, 2020.
- [3] A. J. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, Edinburgh, U.K., 2007, pp. 575–584.
- [4] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proc. ACM SIGOPS 22nd Symp. Operating Systems Principles (SOSP)*, Big Sky, MT, USA, 2009, pp. 117–132.
- [5] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, and J. Li, "Mining invariants from console logs for system problem detection," in *Proc. USENIX Annu. Technical Conf. (USENIX ATC)*, Boston, MA, USA, 2010, pp. 231–244.
- [6] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *Proc. IEEE 27th Int. Symp. Software Reliability Engineering (ISSRE)*, Ottawa, ON, Canada, 2016, pp. 207–218.
- [7] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Dallas, TX, USA, 2017, pp. 1285–1298.
- [8] W. Meng *et al.*, "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artificial Intelligence (IJCAI)*, Macao, China, 2019, pp. 4739–4745.
- [9] X. Zhang *et al.*, "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, Tallinn, Estonia, 2019, pp. 807–817.
- [10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing and Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982–3992.
- [11] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, Shenzhen, China, 2021, pp. 1–8.
- [12] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple contrastive learning of sentence embeddings," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Online and Punta Cana, Dominican Republic, 2021, pp. 6894–6910.

- [13] C. Hu, X. Sun, H. Dai, H. Zhang, and H. Liu, "Research on log anomaly detection based on Sentence-BERT," *Electronics*, vol. 12, no. 17, p. 3580, 2023.
- [14] M. Zhang, J. Chen, J. Liu, J. Wang, R. Shi, and H. Sheng, "LogST: Log semi-supervised anomaly detection based on Sentence-BERT," in *Proc. 7th Int. Conf. Signal and Image Processing (ICSIP)*, Suzhou, China, 2022, pp. 356–361.
- [15] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.
- [16] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Dallas, TX, USA, 2000, pp. 93–104.
- [17] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proc. IEEE*, vol. 77, no. 2, pp. 257–286, Feb. 1989.
- [18] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median," *J. Exp. Social Psychology*, vol. 49, no. 4, pp. 764–766, Jul. 2013.
- [19] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, Art. no. 44, pp. 1–37, Mar. 2014.
- [20] F. J. Massey, "The Kolmogorov–Smirnov test for goodness of fit," *J. Amer. Statistical Assoc.*, vol. 46, no. 253, pp. 68–78, Mar. 1951.
- [21] R. Zang *et al.*, "MLAD: A unified model for multi-system log anomaly detection," arXiv:2401.07655, 2024.